LAPORAN PRAKTIKUM

PEMROGRAMAN BERORIENTASI OBJEK

MODUL 7 – Abstract dan Interface



NAMA: NISA

NIM: 103132400044

KELAS: PS1DS-05-REG-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Abstract class merupakan kelas abstrak dalam Java yang digunakan untuk mendefinisikan kelas umum yang belum memiliki implementasi secara lengkap. Kelas abstrak dapat memiliki atribut, konstruktor, method biasa, dan abstract method yang belum memiliki isi atau implementasi. Kelas abstrak tidak dapat dibuat menjadi objek secara langsung, tetapi harus diturunkan (inheritance) ke kelas lain menggunakan kata kunci `extends`. Kelas turunannya kemudian dapat memberikan implementasi pada abstract method yang diwariskan dari kelas abstrak tersebut.

Interface merupakan sebuah konsep dalam Java yang digunakan untuk mendefinisikan sekumpulan method dan konstanta yang harus diimplementasikan oleh kelas lain. Interface dibuat menggunakan kata kunci `interface`, sedangkan kelas yang menggunakannya menggunakan kata kunci `implements`. Method dalam interface pada dasarnya bersifat public dan abstract, sehingga kelas yang mengimplementasikan interface harus menyediakan isi dari method tersebut. Interface juga mendukung multiple inheritance, yaitu sebuah kelas dapat mengimplementasikan lebih dari satu interface. Pada Java 8, interface juga dapat memiliki default method dan static method.

Abstract class dan interface memiliki fungsi yang hampir sama, yaitu menjadi dasar bagi kelas lain dalam menerapkan konsep abstraksi pada pemrograman berorientasi objek. Namun, keduanya memiliki beberapa perbedaan. Abstract class dapat memiliki atribut, konstruktor, serta method yang sudah memiliki implementasi, sedangkan interface digunakan untuk mendefinisikan karakteristik atau kemampuan yang harus dimiliki suatu kelas. Selain itu, abstract class hanya dapat diwariskan melalui satu kelas induk, sedangkan interface memungkinkan sebuah kelas mengimplementasikan beberapa interface sekaligus. Dengan demikian, pemilihan abstract class atau interface disesuaikan dengan kebutuhan dan struktur program yang dibuat.

B. Guided

a. Guided1

A.Employee

```
package com.nisanisaan.modul07.guided1;


// Fondasi umum: belum punya peran spesifik, jadi tidak boleh diinstansiasi langsung
public abstract class Employee {
    private String name;


    public Employee(String name) {
        this.name = name;
    }


    public String getName() {
        return name;
    }


    // Aturan kosong: setiap pegawai pasti punya gaji, tapi rumusnya beda-beda
    public abstract double earnings();
}
```

B.SalariedEmployee

```
package com.nisanisaan.modul07.guided1;


//wajibb menginput method yang sudah dideklarasikan
public class SalariedEmployee extends Employee {
```

```
    private double weeklySalary;

    public SalariedEmployee(String name, double weeklySalary) {
        super(name);
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings() {
        return weeklySalary;
    }
}
```

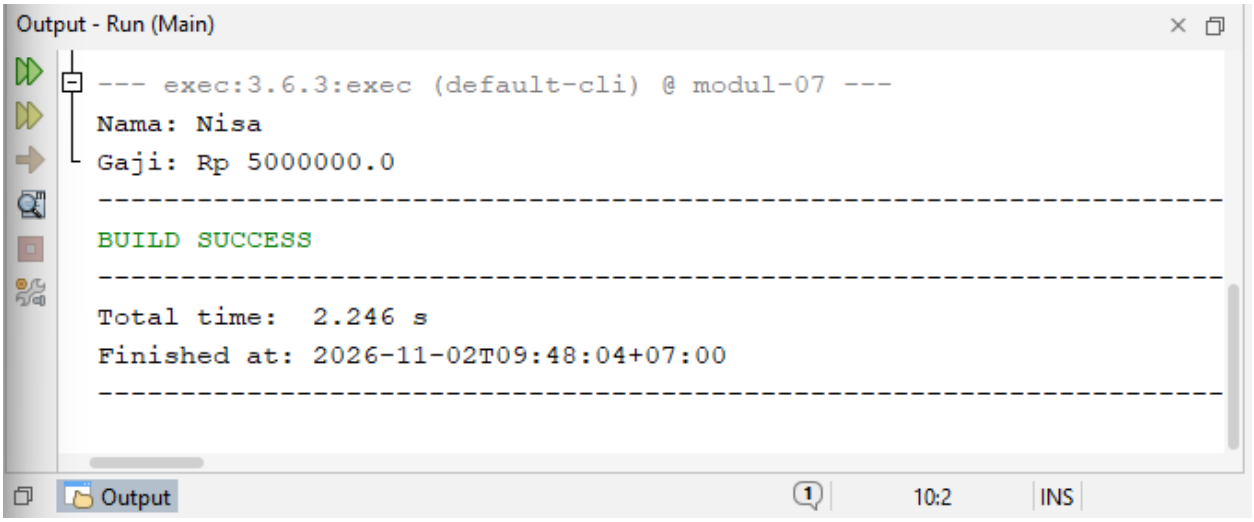
Main.java

```
package com.nisanisaan.modul07.guided1;

public class Main {

    public static void main(String[]args){
        SalariedEmployee salaried = new SalariedEmployee("Nisa", 5000000);
        System.out.println("Nama: "+salaried.getName());
        System.out.println("Gaji: Rp "+ salaried.earnings());
    }
}
```

Output :



b. Guided2

Payable.java

```
package com.nisanisaan.modul07.guided2;

//standar sistem: siapapu
public interface Payable {
    double getPaymentAmount();

}
```

Employee.java

```
package com.nisanisaan.modul07.guided2;

public abstract class Employee implements Payable{ //fondasi umum yang belum
memiliki peran spesifik sehingga tidak boleh diinstansiasi langsung

    private String name;

    //constructur
    public Employee(String name) {
        this.name = name;
    }

    //getter
    public String getName() {
        return name;
    }

    //method, Setiap pegawai pasti punya gaji, tapi rumusnya berbeda-beda
    public abstract double earnings();
```

```
    //saat ada penagihan, alihkan langsung ke metode kalkulasi gaji (earnings)
    @Override
    public double getPaymentAmount(){
        return earnings();
    }
}
```

SalariedEmployee.java

```
package com.nisanisaan.modul07.guided2;

//wajibb menginput method yang sudah dideklarasikan
public class SalariedEmployee extends Employee {

    private double weeklySalary;

    public SalariedEmployee(String name, double weeklySalary) {
        super(name);
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings() {
        return weeklySalary;
    }
}
```

Invoice.java

```
package com.nisanisaan.modul07.guided2;

//Benda mati non-pegawai, tapii tetap diikat oleh aturan
public class Invoice implements Payable {
    private  String partName;
    private int quantity;
    private double pricePerItem;

    public Invoice(String partName, int quantity, double pricePerItem){
        this.partName = partName;
        this.quantity = quantity;
        this.pricePerItem = pricePerItem;
    }

    public String getPartName(){
        return partName;
    }

    @Override
```

```
    public double getPaymentAmount(){
        return quantity * pricePerItem;
    }
}
```

Main.java

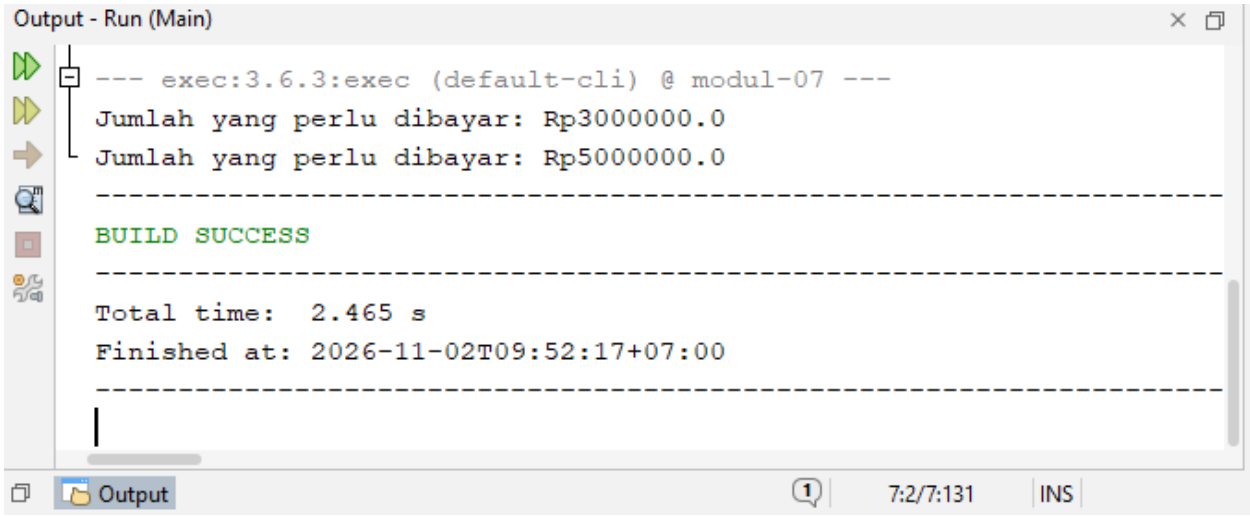
```
package com.nisanisaan.modul07.guided2;

public class Main {
    public static void main(String[]args){
        Payable[] items = new Payable[2];
        items[0] = new Invoice("Monitor LED", 2, 1500000);
        items[1] = new SalariedEmployee("Nisa", 5000000);

        //Dynamic dispatch: Java otomatis mengeksekusi rumus yang tepat
        for (Payable item : items){
            System.out.println("Jumlah yang perlu dibayar: Rp" + item.getPaymentAmount());

        }
    }
}
```

Output :



c. Guided3

Payable.java

```
package com.nisanisaan.modul07.guided3;

public interface Payable {
    double getPaymentAmount();

    default void printReceipt(){
        System.out.println("Struk Resmi - Total: Rp" + getPaymentAmount());
    }
    static void logTransaction(String message){
        System.out.println("[AUDIT LOG]" + message);
    }
}
```

Employee.java

```
package com.nisanisaan.modul07.guided3;

public abstract class Employee implements Payable{ //fondasi umum yang belum
memiliki peran spesifik sehingga tidak boleh diinstansiasi langsung

    private String name;

    //constructur
    public Employee(String name) {
        this.name = name;
    }

    //getter
    public String getName() {
        return name;
    }

    //method, Setiap pegawai pasti punya gaji, tapi rumusnya berbeda-beda
    public abstract double earnings();

    //saat ada penagihan, alihkan langsung ke metode kalkulasi gaji (earnings)
    @Override
    public double getPaymentAmount(){
        return earnings();
    }
}
```

SalariedEmployee.java

```
package com.nisanisaan.modul07.guided3;
```

```
//wajibb menginput method yang sudah dideklarasikan
public class SalariedEmployee extends Employee {

    private double weeklySalary;

    public SalariedEmployee(String name, double weeklySalary) {
        super(name);
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings() {
        return weeklySalary;
    }
}
```

Invoice.java

```
package com.nisanisaan.modul07.guided3;

//Benda mati non-pegawai, tapii tetap diikat oleh aturan
public class Invoice implements Payable {
    private  String partName;
    private int quantity;
    private double pricePerItem;

    public Invoice(String partName, int quantity, double pricePerItem){
        this.partName = partName;
        this.quantity = quantity;
        this.pricePerItem = pricePerItem;
    }

    public String getPartName(){
        return partName;
    }

    @Override
    public double getPaymentAmount(){
        return quantity * pricePerItem;
    }
}
```

Main.java

```
package com.nisanisaan.modul07.guided3;


public class Main {
```

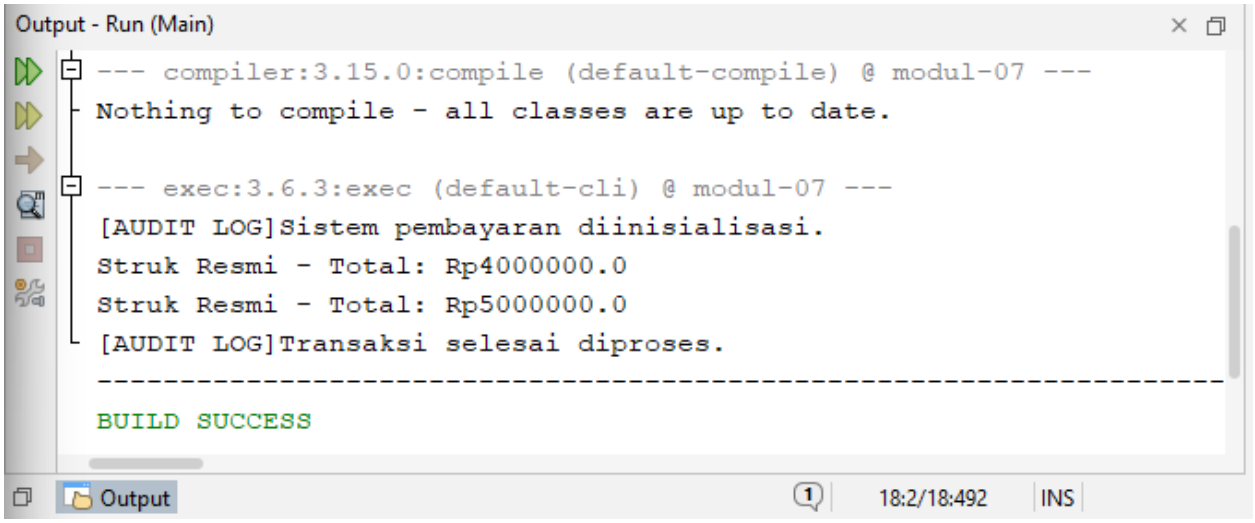
```
    public static void main(String[] args) {
        Payable.logTransaction("Sistem pembayaran diinisialisasi.");

        Payable[] items = new Payable[2];
        items[0] = new Invoice("Ram DDR 5 16GB", 2, 2000000);
        items[1] = new SalariedEmployee("Andita", 5000000);

        for (Payable item : items){
            item.printReceipt();
        }

        Payable.logTransaction("Transaksi selesai diproses.");
    }
}
```

Output :



C. Unguided

1. Soal Unguided

Soal ini meminta kamu membuat program Java sistem kasir bioskop (Cinema POS) yang memproses tiket nonton dan paket snack F&B dalam satu keranjang belanja, dengan menerapkan prinsip OOP. Semua kelas dan interface (`ItemTransaksi`, `TiketBioskop`, `TiketReguler`, `TiketVIP`, `PaketSnack`, dan `Main`) harus

berada di direktori `unguided/`, satu file per kelas, dengan baris `package` di bagian atas (misalnya `com.[nama_panggilan].unguided`). Semua field wajib `private` dan hanya diakses lewat getter, konstruktor harus memakai parameter bernama sama dengan field sehingga menggunakan `this`, dan dilarang memakai koleksi dinamis seperti `ArrayList`, jadi gunakan array statis saja.

Pada Tugas 1, buat kelas abstrak `TiketBioskop` dengan atribut `judulFilm`, `hargaDasar`, dan `jumlahKursi`, lengkap dengan konstruktor, getter, serta dua metode abstrak `hitungTotalBayar()` dan `getNamaTransaksi()`. Kelas ini diturunkan menjadi `TiketReguler`, yang memiliki atribut tambahan `nomorStudio` dan menghitung total sebagai `hargaDasar * jumlahKursi`, serta `TiketVIP`, yang memiliki atribut tambahan `biayaFasilitasVIP` per kursi sehingga totalnya `(hargaDasar + biayaFasilitasVIP) * jumlahKursi`. Masing-masing subclass juga harus mengembalikan string nama transaksi dengan format tertentu yang memuat jenis tiket, judul film, jumlah kursi, dan harga satuan.

Pada Tugas 2, buat interface `ItemTransaksi` dengan dua kontrak, yaitu `getNominalBayar()` dan `getNamaTransaksi()`. `TiketBioskop` harus mengimplementasikan interface ini, dengan `getNominalBayar()` cukup mengembalikan hasil `hitungTotalBayar()`. Lalu buat kelas independen `PaketSnack` (bukan turunan `TiketBioskop`) dengan atribut `namaPaket`, `jumlahPorsi`, dan `hargaPerPorsi`, yang juga mengimplementasikan `ItemTransaksi`, di mana nominal bayarnya adalah `jumlahPorsi * hargaPerPorsi` dan nama transaksinya berformat `"F&B Snack: ..."`.

Pada Tugas 3, tambahkan fitur Java 8+ ke interface, yaitu default method `cetakRincianItem()` yang mencetak satu baris struk dengan format `- [Nama Transaksi] | Total: Rp[Nominal Bayar]` memakai data dari `getNamaTransaksi()` dan `getNominalBayar()`, serta static method `catatLogKasir(String pesan)` yang mencetak `[LOG SISTEM KASIR] [pesan]`, tanpa perlu mengubah kelas-kelas lain.

Pada Tugas 4, di dalam `main` kamu harus menuliskan baris instansiasi `new TiketBioskop(...)` lalu menjadikannya komentar untuk membuktikan kelas abstrak tidak bisa diinstansiasi, kemudian memanggil `ItemTransaksi.catatLogKasir("Membuka transaksi baru untuk Pelanggan #042.")`. Setelah itu buat tiga objek: `TiketReguler` (Interstellar,

harga 40000, 2 kursi, Studio 3), `TiketVIP` (Dune: Part Two, harga 60000, 2 kursi, biaya VIP 25000), dan `PaketSnack` (Combo Popcorn Caramel + 2 Soda, 1 porsi, harga 55000), lalu simpan ketiganya dalam array heterogen bertipe `ItemTransaksi[]`. Lakukan perulangan pada array tersebut untuk memanggil `cetakRincianItem()` di setiap elemen sambil menjumlahkan nominal bayarnya, tampilkan grand total, lalu tutup dengan `catatLogKasir("Transaksi #042 selesai diproses.")`. Output yang diharapkan berupa log pembuka, daftar tiga item beserta totalnya masing-masing (Rp80000.0, Rp170000.0, dan Rp55000.0), total transaksi Rp305000.0, dan log penutup.

Jawaban :

ItemTransaksi.java

```
package com.nisanisaan.modul07.unguided;


public interface ItemTransaksi {
    double getNominalBayar();
    String getNamaTransaksi();


    default void cetakRincianItem() {
        System.out.println("- " + getNamaTransaksi() + " | Total: Rp" + getNominalBayar());
    }


    static void catatLogKasir(String pesan) {
        System.out.println("[LOG SISTEM KASIR] " + pesan);
    }
}
```

TiketBioskop.java

```
package com.nisanisaan.modul07.unguided;
```

```
public abstract class TiketBioskop implements ItemTransaksi {
    private String judulFilm;
    private double hargaDasar;
    private int jumlahKursi;

    public TiketBioskop(String judulFilm, double hargaDasar, int jumlahKursi) {
        this.judulFilm = judulFilm;
        this.hargaDasar = hargaDasar;
        this.jumlahKursi = jumlahKursi;
    }

    public String getJudulFilm() {
        return judulFilm;
    }

    public double getHargaDasar() {
        return hargaDasar;
    }

    public int getJumlahKursi() {
        return jumlahKursi;
    }

    public abstract double hitungTotalBayar();

    @Override
    public abstract String getNamaTransaksi();
```

```
    @Override
    public double getNominalBayar() {
        return hitungTotalBayar();
    }
}
```

TiketReguler.java

```
package com.nisanisaan.modul07.unguided;


public class TiketReguler extends TiketBioskop {
    private int nomorStudio;


    public TiketReguler(String judulFilm, double hargaDasar, int jumlahKursi, int
nomorStudio) {
        super(judulFilm, hargaDasar, jumlahKursi);
        this.nomorStudio = nomorStudio;
    }


    @Override
    public double hitungTotalBayar() {
        return getHargaDasar() * getJumlahKursi();
    }


    @Override
    public String getNamaTransaksi() {
        return "Tiket Reguler (Studio " + nomorStudio + ") - " + getJudulFilm()
                + " (" + getJumlahKursi() + " x Rp" + getHargaDasar() + ")";
```

```
    }
}
```

TiketVIP.java

```
package com.nisanisaan.modul07.unguided;


public class TiketVIP extends TiketBioskop {
    private double biayaFasilitasVIP;


    public TiketVIP(String judulFilm, double hargaDasar, int jumlahKursi, double
biayaFasilitasVIP) {
        super(judulFilm, hargaDasar, jumlahKursi);
        this.biayaFasilitasVIP = biayaFasilitasVIP;
    }


    @Override
    public double hitungTotalBayar() {
        return (getHargaDasar() + biayaFasilitasVIP) * getJumlahKursi();
    }


    @Override
    public String getNamaTransaksi() {
        return "Tiket VIP Premiere - " + getJudulFilm()
                + " (" + getJumlahKursi() + " x Rp" + (getHargaDasar() + biayaFasilitasVIP) +
")";
    }
}
```

PaketSnack.java

```
package com.nisanisaan.modul07.unguided;


public class PaketSnack implements ItemTransaksi {
    private String namaPaket;
    private int jumlahPorsi;
    private double hargaPerPorsi;


    public PaketSnack(String namaPaket, int jumlahPorsi, double hargaPerPorsi) {
        this.namaPaket = namaPaket;
        this.jumlahPorsi = jumlahPorsi;
        this.hargaPerPorsi = hargaPerPorsi;
    }


    @Override
    public double getNominalBayar() {
        return jumlahPorsi * hargaPerPorsi;
    }


    @Override
    public String getNamaTransaksi() {
        return "F&B Snack: " + namaPaket + " (" + jumlahPorsi + " x Rp" + hargaPerPorsi +
")";
    }
}
```

Main.java

```
package com.nisanisaan.modul07.unguided;
```

```
public class Main {
    public static void main(String[] args) {
        // 1. Uji proteksi abstraksi (error jika tidak dikomentari)
        // TiketBioskop t = new TiketBioskop("Test", 10000, 1);

        // 2. Buka kasir
        ItemTransaksi.catatLogKasir("Membuka transaksi baru untuk Pelanggan #042.");

        // 3. Buat objek transaksi
        TiketReguler reguler = new TiketReguler("Interstellar", 40000, 2, 3);
        TiketVIP vip = new TiketVIP("Dune: Part Two", 60000, 2, 25000);
        PaketSnack snack = new PaketSnack("Combo Popcorn Caramel + 2 Soda", 1,
55000);

        // 4. Array polimorfik
        ItemTransaksi[] keranjang = { reguler, vip, snack };

        // 5. Pemrosesan dinamis
        System.out.println("=== DAFTAR TRANSAKSI BIOSKOP ===");
        double grandTotal = 0;
        for (ItemTransaksi item : keranjang) {
            item.cetakRincianItem();
            grandTotal += item.getNominalBayar();
        }

        // 6. Tutup kasir
        System.out.println("-----------------------------------------------------------");
        System.out.println("TOTAL TRANSAKSI : Rp" + grandTotal);
```

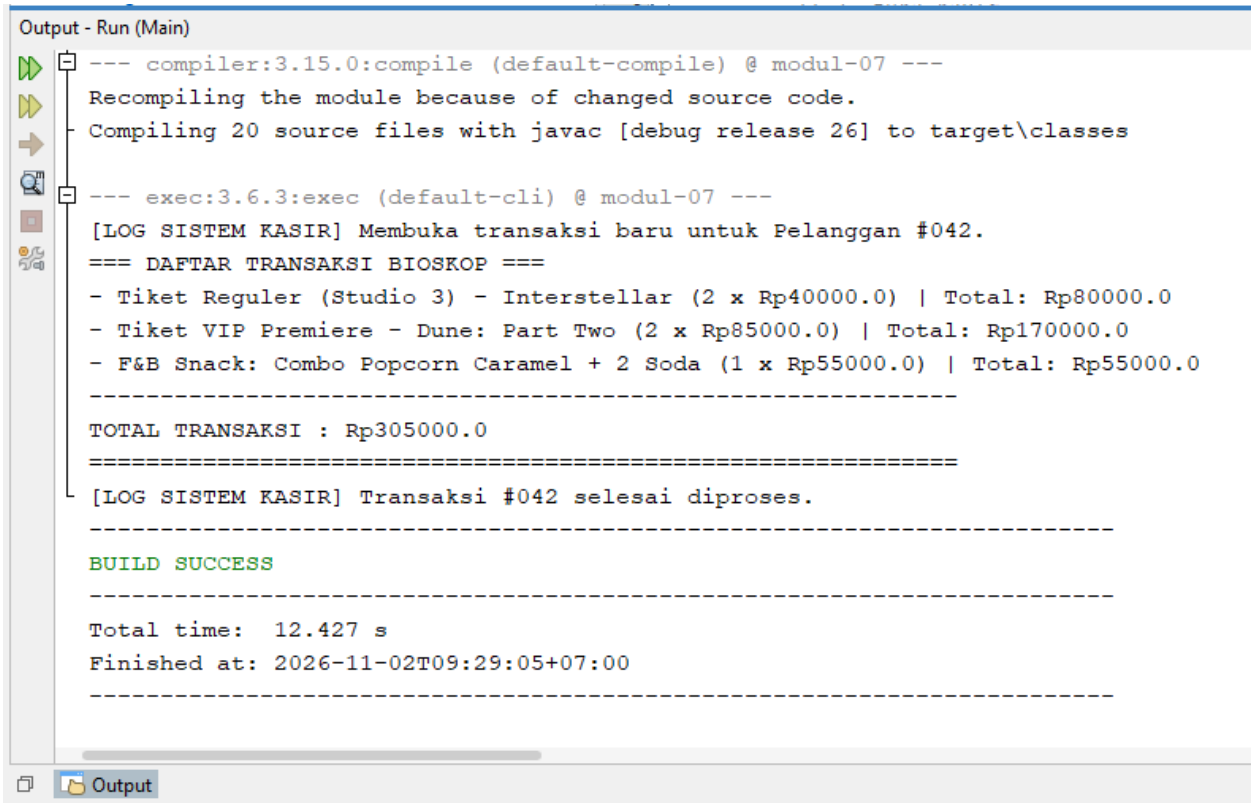
```
System.out.println("============================================================");

        ItemTransaksi.catatLogKasir("Transaksi #042 selesai diproses.");

    }

}
```

Output :



Penjelasan :

Proyek ini adalah sistem kasir bioskop sederhana berbasis Java yang memperlihatkan konsep abstraksi, interface, dan polimorfisme. Interface `ItemTransaksi` menjadi kontrak umum (nominal bayar dan nama transaksi) sehingga tiket dan makanan, meski berasal dari hierarki berbeda, bisa diperlakukan sama oleh kasir. `TiketBioskop` adalah abstract class yang menyimpan data dasar tiket (judul film, harga, jumlah kursi) dan mewajibkan turunannya, yaitu `TiketReguler` dan

`TiketVIP`, menghitung total bayar dengan cara masing-masing (VIP ditambah biaya fasilitas per kursi), sedangkan `PaketSnack` mengimplementasikan interface yang sama secara independen. Berkat default method `cetakRincianItem()` dan static method `catatLogKasir()` (fitur Java 8+), `Main` cukup menyimpan semua objek dalam satu array bertipe `ItemTransaksi[]`, mencetak struk lewat perulangan, dan menjumlahkan grand total tanpa perlu tahu jenis item aslinya. Seluruh field bersifat private dengan akses lewat getter, konstruktor memakai `this`, dan koleksi menggunakan array statis sesuai ketentuan soal.

D. Kesimpulan

Berdasarkan pembelajaran Modul 7, dapat disimpulkan bahwa abstract class dan interface merupakan konsep penting dalam pemrograman berorientasi objek menggunakan Java. Melalui modul ini, praktikan dapat memahami cara membuat kelas abstrak, menerapkan abstract method, serta menggunakan interface pada sebuah kelas. Selain itu, praktikan juga dapat memahami perbedaan antara abstract class dan interface, baik dari segi penggunaan maupun penerapannya dalam program. Dengan mempelajari kedua konsep tersebut, praktikan dapat membuat program yang lebih terstruktur, fleksibel, dan mudah dikembangkan.